8 - Texto y archivos

Alejandro Veloz

aavelozb.github.io/py



Objetivos de la unidad

- **Procesar texto** con `strip`, `replace`, `split`, `join` y `format`.
- **Leer** un archivo de texto línea a línea y cargar sus datos en listas y diccionarios.
- **Generar información** a partir de los datos de uno o más archivos.
- **Escribir** resultados en un archivo, con un formato especificado.

El problema

Los datos de un programa viven en la memoria mientras el programa corre.

```
notas = {'Ana': 55, 'Luis': 70, 'Paula': 48}
```

El problema

Los datos de un programa viven en la memoria mientras el programa corre.

```
notas = {'Ana': 55, 'Luis': 70, 'Paula': 48}
```

- Al terminar el programa, el diccionario **desaparece**.
- Ingresar miles de registros con `input` en cada ejecución no es viable.

El problema

Los datos de un programa viven en la memoria mientras el programa corre.

```
notas = {'Ana': 55, 'Luis': 70, 'Paula': 48}
```

- Al terminar el programa, el diccionario **desaparece**.
- Ingresar miles de registros con `input` en cada ejecución no es viable.

Un archivo guarda los datos en el disco. Otro programa puede leerlos después.

Un archivo de texto

El archivo `inmigrantes.txt` registra los ingresos al país:

```
456:Cristina Palermo:Argentina:07-05-2019
523:Sophia Watson:EEUU:02-05-2019
364:Alan Moore:EEUU:12-12-2018
111:Karen Lee:EEUU:05-09-2019
646:Nicolás Cabrera:Venezuela:01-01-2019
```

Un archivo de texto

El archivo `inmigrantes.txt` registra los ingresos al país:

```
456:Cristina Palermo:Argentina:07-05-2019
523:Sophia Watson:EEUU:02-05-2019
364:Alan Moore:EEUU:12-12-2018
111:Karen Lee:EEUU:05-09-2019
646:Nicolás Cabrera:Venezuela:01-01-2019
```

- Cada **línea** es un registro: identificador, nombre, país y fecha.
- Los **campos** se separan con un carácter fijo, aquí `:`.
- Un archivo **CSV** (*comma separated values*) tiene la misma forma, con `,` o `;` como separador.

Caracteres especiales

Fin de línea y tabulación

Dentro de un string, `\n` es un **fin de línea** y `\t` una **tabulación**.

```
print('Ana\t55\nLuis\t70')
```

Fin de línea y tabulación

Dentro de un string, `\n` es un **fin de línea** y `\t` una **tabulación**.

```
print('Ana\t55\nLuis\t70')
```

```
Ana     55
Luis    70
```

Fin de línea y tabulación

Dentro de un string, `\n` es un **fin de línea** y `\t` una **tabulación**.

```
print('Ana\t55\nLuis\t70')
```

```
Ana     55
Luis    70
```

- `\n` y `\t` se escriben con dos símbolos, pero son **un solo carácter**.

```
print(len('Ana\t55'))      # 6
```

Espacios en los bordes con `strip`

`s.strip()` retorna una copia de `s` sin espacios, `\n` ni `\t` al inicio y al final.

```
s = '  Chile \n'
t = s.strip()
print(len(s), len(t))     # 9 5
print(t == 'Chile')       # True
print(s == 'Chile')       # False
```

Espacios en los bordes con `strip`

`s.strip()` retorna una copia de `s` sin espacios, `\n` ni `\t` al inicio y al final.

```
s = '  Chile \n'
t = s.strip()
print(len(s), len(t))     # 9 5
print(t == 'Chile')       # True
print(s == 'Chile')       # False
```

- Los espacios interiores se conservan. `' San Pedro '.strip()` es `'San Pedro'`.
- `s` no cambia, porque los strings son **inmutables**.

Métodos de strings

Reemplazo con `replace`

`s.replace(a, b)` retorna una copia de `s` con cada `a` cambiado por `b`.

```
visitas = '89.000.331'
print(visitas.replace('.', ''))            # 89000331
print(int(visitas.replace('.', '')) + 1)   # 89000332

nota = '5,6'
print(float(nota.replace(',', '.')))       # 5.6
```

Reemplazo con `replace`

`s.replace(a, b)` retorna una copia de `s` con cada `a` cambiado por `b`.

```
visitas = '89.000.331'
print(visitas.replace('.', ''))            # 89000331
print(int(visitas.replace('.', '')) + 1)   # 89000332

nota = '5,6'
print(float(nota.replace(',', '.')))       # 5.6
```

`replace` no modifica `visitas`, sino que retorna un string nuevo.

Separación con `split`

`s.split(sep)` corta `s` en cada `sep` y retorna una **lista de strings**.

```
linea = '456:Cristina Palermo:Argentina:07-05-2019'
campos = linea.split(':')
print(campos)
# ['456', 'Cristina Palermo', 'Argentina', '07-05-2019']
print(campos[2])      # Argentina
```

Separación con `split`

`s.split(sep)` corta `s` en cada `sep` y retorna una **lista de strings**.

```
linea = '456:Cristina Palermo:Argentina:07-05-2019'
campos = linea.split(':')
print(campos)
# ['456', 'Cristina Palermo', 'Argentina', '07-05-2019']
print(campos[2])      # Argentina
```

- El separador no aparece en el resultado.
- Todos los elementos son strings, también `'456'`.

`split` sin argumento

Sin argumento, `split` corta en cualquier secuencia de espacios, `\t` o `\n`.

```
notas = '5.6  6.5\t6.6\n'
print(notas.split())        # ['5.6', '6.5', '6.6']
print(notas.split(' '))     # ['5.6', '', '6.5\t6.6\n']
```

`split` sin argumento

Sin argumento, `split` corta en cualquier secuencia de espacios, `\t` o `\n`.

```
notas = '5.6  6.5\t6.6\n'
print(notas.split())        # ['5.6', '6.5', '6.6']
print(notas.split(' '))     # ['5.6', '', '6.5\t6.6\n']
```

- Con `' '` como separador, dos espacios seguidos producen un string vacío.
- Para separar palabras o números escritos con espacios, se usa `split()` sin argumento.

Conversión de los campos

```
texto = '5.6 6.5 6.6'
notas = []
for x in texto.split():
    notas.append(float(x))
print(notas)                    # [5.6, 6.5, 6.6]
print(sum(notas) / len(notas))  # 6.233333333333333
```

Conversión de los campos

```
texto = '5.6 6.5 6.6'
notas = []
for x in texto.split():
    notas.append(float(x))
print(notas)                    # [5.6, 6.5, 6.6]
print(sum(notas) / len(notas))  # 6.233333333333333
```

Sin la conversión, `sum` produce `TypeError`, porque los elementos de `texto.split()` son strings.

Desempaquetado de los campos

```
linea = 'UA7998;Tokyo;12:00'
vuelo, destino, hora = linea.split(';')
print(destino)                    # Tokyo

hh, mm = hora.split(':')
print(int(hh) * 60 + int(mm))     # 720
```

Desempaquetado de los campos

```
linea = 'UA7998;Tokyo;12:00'
vuelo, destino, hora = linea.split(';')
print(destino)                    # Tokyo

hh, mm = hora.split(':')
print(int(hh) * 60 + int(mm))     # 720
```

La cantidad de variables debe coincidir con la cantidad de campos. Si no coincide, se produce `ValueError`.

Varios separadores en una línea

El itinerario de un vuelo usa `:` y `/` en la misma línea.

```
linea = 'NAC23:Santiago/Iquique/1600'
codigo, ruta = linea.split(':')
origen, destino, km = ruta.split('/')
print(codigo, origen, destino, int(km))
# NAC23 Santiago Iquique 1600
```

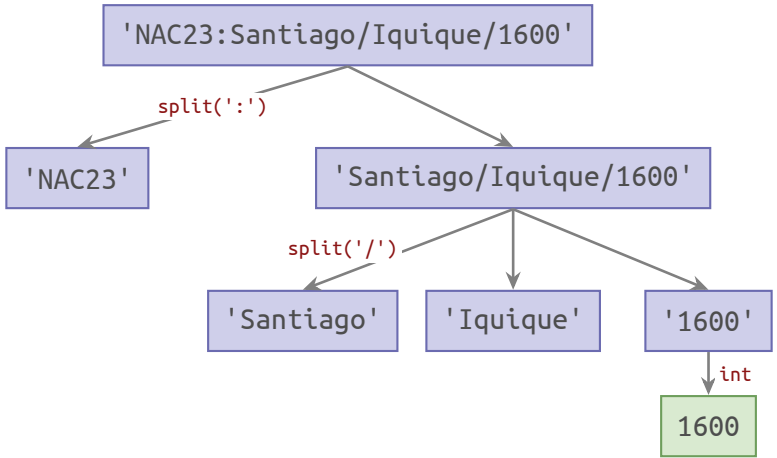
Varios separadores en una línea

El itinerario de un vuelo usa `:` y `/` en la misma línea.

```
linea = 'NAC23:Santiago/Iquique/1600'
codigo, ruta = linea.split(':')
origen, destino, km = ruta.split('/')
print(codigo, origen, destino, int(km))
# NAC23 Santiago Iquique 1600
```

- Primero se corta por el separador **externo**.
- Después, cada parte se corta por su separador **interno**.

Separación en dos niveles



Los strings van en azul y el número en verde. Sólo el último paso cambia el tipo.

Unión con `join`

`sep.join(lista)` une los strings de `lista` y pone `sep` entre cada par.

```
ciudades = ['Santiago', 'Iquique', 'Arica']
print('/'.join(ciudades))      # Santiago/Iquique/Arica
print(', '.join(ciudades))     # Santiago, Iquique, Arica
```

Unión con `join`

`sep.join(lista)` une los strings de `lista` y pone `sep` entre cada par.

```
ciudades = ['Santiago', 'Iquique', 'Arica']
print('/'.join(ciudades))      # Santiago/Iquique/Arica
print(', '.join(ciudades))     # Santiago, Iquique, Arica
```

- `join` hace la operación inversa de `split`.
- Todos los elementos deben ser strings. `','.join([5.6, 6.5])` produce `TypeError`.

Plantillas con `format`

`format` reemplaza cada {} de una plantilla por un valor, en orden.

```
nombre = 'Ana'
puntos = 62
print('{} obtuvo {} puntos'.format(nombre, puntos))
# Ana obtuvo 62 puntos
```

Plantillas con `format`

`format` reemplaza cada {} de una plantilla por un valor, en orden.

```
nombre = 'Ana'
puntos = 62
print('{} obtuvo {} puntos'.format(nombre, puntos))
# Ana obtuvo 62 puntos
```

- Los valores pueden ser de cualquier tipo, sin `str()`.
- `{:.1f}` muestra un real con un decimal.

```
print('{:.1f}%'.format(200 / 3))      # 66.7%
```

Concatenación y plantilla

Las dos líneas construyen el mismo string:

```
linea = vuelo + ';' + destino + ';' + str(minutos) + '\n'
linea = '{};{};{}\n'.format(vuelo, destino, minutos)
```

Concatenación y plantilla

Las dos líneas construyen el mismo string:

```
linea = vuelo + ';' + destino + ';' + str(minutos) + '\n'
linea = '{};{};{}\n'.format(vuelo, destino, minutos)
```

- Con `+`, cada número necesita `str()`.
- La plantilla muestra el formato completo de la línea, con sus separadores y el `\n` final.

Operaciones sobre strings

Operación	Resultado
`s.strip()`	Copia sin espacios, `\n` ni `\t` en los bordes
`s.replace(a, b)`	Copia con cada `a` cambiado por `b`
`s.split(sep)`	Lista de strings separados por `sep`
`s.split()`	Lista de palabras separadas por espacios
`sep.join(l)`	String con los elementos de `l` unidos por `sep`
`p.format(x, y)`	Copia de `p` con cada `{}` reemplazado

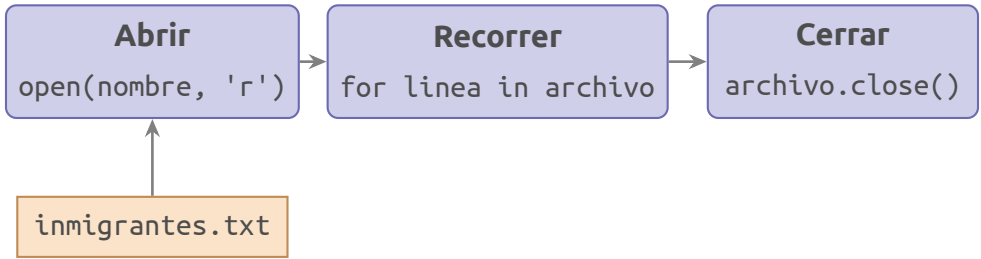
Operaciones sobre strings

Operación	Resultado
`s.strip()`	Copia sin espacios, `\n` ni `\t` en los bordes
`s.replace(a, b)`	Copia con cada `a` cambiado por `b`
`s.split(sep)`	Lista de strings separados por `sep`
`s.split()`	Lista de palabras separadas por espacios
`sep.join(l)`	String con los elementos de `l` unidos por `sep`
`p.format(x, y)`	Copia de `p` con cada `{}` reemplazado

Ninguno modifica el string original. Todos retornan un valor nuevo.

Lectura de archivos

Protocolo de uso



- `open` retorna un **objeto archivo**, conectado al archivo del disco.
- `close` termina esa conexión. Se cierra cuando ya no se necesita.

Modos de apertura

Modo	Uso	Si no existe	Si ya existe
`'r'`	Leer	`FileNotFoundError`	Se lee desde el inicio
`'w'`	Escribir	Se crea	Se borra su contenido
`'a'`	Agregar	Se crea	Se escribe al final

Modos de apertura

Modo	Uso	Si no existe	Si ya existe
`'r'`	Leer	`FileNotFoundError`	Se lee desde el inicio
`'w'`	Escribir	Se crea	Se borra su contenido
`'a'`	Agregar	Se crea	Se escribe al final

- `open(nombre)` sin modo equivale a `open(nombre, 'r')`.
- El nombre incluye la extensión: `'inmigrantes.txt'`, no `'inmigrantes'`.

Lectura línea a línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    print(linea)
archivo.close()
```

Lectura línea a línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    print(linea)
archivo.close()
```

```
456:Cristina Palermo:Argentina:07-05-2019

523:Sophia Watson:EEUU:02-05-2019

364:Alan Moore:EEUU:12-12-2018
...
```

Lectura línea a línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    print(linea)
archivo.close()
```

```
456:Cristina Palermo:Argentina:07-05-2019

523:Sophia Watson:EEUU:02-05-2019

364:Alan Moore:EEUU:12-12-2018
...
```

Cada línea termina en `\n`, y `print` agrega otro salto. Por eso aparece una línea en blanco entre registros.

Limpiar y separar cada línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    campos = linea.strip().split(':')
    print(campos[1], 'viene de', campos[2])
archivo.close()
```

Limpiar y separar cada línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    campos = linea.strip().split(':')
    print(campos[1], 'viene de', campos[2])
archivo.close()
```

```
Cristina Palermo viene de Argentina
Sophia Watson viene de EEUU
Alan Moore viene de EEUU
Karen Lee viene de EEUU
Nicolás Cabrera viene de Venezuela
```

Limpiar y separar cada línea

```
archivo = open('inmigrantes.txt', 'r')
for linea in archivo:
    campos = linea.strip().split(':')
    print(campos[1], 'viene de', campos[2])
archivo.close()
```

```
Cristina Palermo viene de Argentina
Sophia Watson viene de EEUU
Alan Moore viene de EEUU
Karen Lee viene de EEUU
Nicolás Cabrera viene de Venezuela
```

`linea.strip().split(sep)` es la primera instrucción del ciclo en casi todos los problemas con archivos.

Una línea paso a paso

Expresión	Valor
`linea`	`'364:Alan Moore:EEUU:12-12-2018\n'`

Una línea paso a paso

Expresión	Valor
`linea`	`'364:Alan Moore:EEUU:12-12-2018\n'`
`linea.strip()`	`'364:Alan Moore:EEUU:12-12-2018'`

Una línea paso a paso

Expresión	Valor
`linea`	`'364:Alan Moore:EEUU:12-12-2018\n'`
`linea.strip()`	`'364:Alan Moore:EEUU:12-12-2018'`
`campos`	`['364', 'Alan Moore', 'EEUU', '12-12-2018']`

Una línea paso a paso

Expresión	Valor
`linea`	`'364:Alan Moore:EEUU:12-12-2018\n'`
`linea.strip()`	`'364:Alan Moore:EEUU:12-12-2018'`
`campos`	`['364', 'Alan Moore', 'EEUU', '12-12-2018']`
`campos[3].split('-')`	`['12', '12', '2018']`

Una línea paso a paso

Expresión	Valor
`linea`	`'364:Alan Moore:EEUU:12-12-2018\n'`
`linea.strip()`	`'364:Alan Moore:EEUU:12-12-2018'`
`campos`	`['364', 'Alan Moore', 'EEUU', '12-12-2018']`
`campos[3].split('-')`	`['12', '12', '2018']`
`int(campos[3].split('-')[2])`	`2018`

Cada paso aplica una sola operación al resultado del paso anterior.

Del archivo a un diccionario

Cantidad de personas que ingresaron desde cada país:

```
archivo = open('inmigrantes.txt', 'r')
cuenta = {}
for linea in archivo:
    campos = linea.strip().split(':')
    pais = campos[2]
    if pais not in cuenta:
        cuenta[pais] = 0
    cuenta[pais] += 1
archivo.close()
print(cuenta)   # {'Argentina': 1, 'EEUU': 3, 'Venezuela': 1}
```

Del archivo a un diccionario

Cantidad de personas que ingresaron desde cada país:

```
archivo = open('inmigrantes.txt', 'r')
cuenta = {}
for linea in archivo:
    campos = linea.strip().split(':')
    pais = campos[2]
    if pais not in cuenta:
        cuenta[pais] = 0
    cuenta[pais] += 1
archivo.close()
print(cuenta)   # {'Argentina': 1, 'EEUU': 3, 'Venezuela': 1}
```

Es el conteo por categoría, con las líneas del archivo como datos.

Una función que lee un archivo

```
def filtrar(nombre_archivo, año):
    archivo = open(nombre_archivo, 'r')
    dicc = {}
    for linea in archivo:
        campos = linea.strip().split(':')
        fecha = campos[3].split('-')
        if int(fecha[2]) == año:
            pais = campos[2]
            if pais not in dicc:
                dicc[pais] = []
            dicc[pais].append(campos[1])
    archivo.close()
    return dicc
```

Resultado de `filtrar`

`filtrar` asocia cada país con los nombres de quienes ingresaron el año indicado.

```
>>> filtrar('inmigrantes.txt', 2019)
{'Argentina': ['Cristina Palermo'],
 'EEUU': ['Sophia Watson', 'Karen Lee'],
 'Venezuela': ['Nicolás Cabrera']}
```

Resultado de `filtrar`

`filtrar` asocia cada país con los nombres de quienes ingresaron el año indicado.

```
>>> filtrar('inmigrantes.txt', 2019)
{'Argentina': ['Cristina Palermo'],
 'EEUU': ['Sophia Watson', 'Karen Lee'],
 'Venezuela': ['Nicolás Cabrera']}
```

- El **nombre del archivo** es un parámetro, igual que el año.
- El archivo se abre y se cierra **dentro** de la función.
- Alan Moore no aparece, porque ingresó en 2018.

Dos archivos relacionados

`alumnos.txt` asocia cada RUT con un nombre. Cada alumno tiene además un archivo `RUT.txt` con sus ramos y notas.

`alumnos.txt`

```
150870162:Juan Duran
167754321:Lorena Perez
156432876:Alvaro Ruiz
167844261:Marcelo Pinto
```

`150870162.txt`

```
INS-131,5.6 6.5 6.6
MAT-100,5.8 1.4 1.5 4.1
FIS-110,1.0 6.3
INS-141,4.9 5.6 6.3 4.3 4.9
```

Dos archivos relacionados

`alumnos.txt` asocia cada RUT con un nombre. Cada alumno tiene además un archivo `RUT.txt` con sus ramos y notas.

`alumnos.txt`

```
150870162:Juan Duran
167754321:Lorena Perez
156432876:Alvaro Ruiz
167844261:Marcelo Pinto
```

`150870162.txt`

```
INS-131,5.6 6.5 6.6
MAT-100,5.8 1.4 1.5 4.1
FIS-110,1.0 6.3
INS-141,4.9 5.6 6.3 4.3 4.9
```

- En `RUT.txt`, la `,` separa el ramo de las notas, y los espacios separan las notas.
- RUT es el rol único tributario.

Notas de un alumno

```
def notas(rut):
    archivo = open(rut + '.txt', 'r')
    ramos = {}
    for linea in archivo:
        ramo, texto = linea.strip().split(',')
        ramos[ramo] = []
        for x in texto.split():
            ramos[ramo].append(float(x))
    archivo.close()
    return ramos
```

Notas de un alumno

```
def notas(rut):
    archivo = open(rut + '.txt', 'r')
    ramos = {}
    for linea in archivo:
        ramo, texto = linea.strip().split(',')
        ramos[ramo] = []
        for x in texto.split():
            ramos[ramo].append(float(x))
    archivo.close()
    return ramos
```

```
>>> notas('150870162')
{'INS-131': [5.6, 6.5, 6.6], 'MAT-100': [5.8, 1.4, 1.5, 4.1],
 'FIS-110': [1.0, 6.3], 'INS-141': [4.9, 5.6, 6.3, 4.3, 4.9]}
```

Mejor alumno de un ramo

```
def mejor_alumno(ramo):
    archivo = open('alumnos.txt', 'r')
    mejor = ''
    mayor = 0
    for linea in archivo:
        rut, nombre = linea.strip().split(':')
        ramos = notas(rut)
        if ramo in ramos:
            prom = sum(ramos[ramo]) / len(ramos[ramo])
            if prom > mayor:
                mayor = prom
                mejor = nombre
    archivo.close()
    return mejor
```

Resultado de `mejor_alumno`

Promedios en INS-131:

Alumno	Notas	Promedio
Juan Duran	5.6 6.5 6.6	6,23
Lorena Perez	5.5 6.5	6,00
Alvaro Ruiz	3.5 5.0 5.6 2.3	4,10
Marcelo Pinto	5.5 5.5 6.6 6.3	5,98

Resultado de `mejor_alumno`

Promedios en INS-131:

Alumno	Notas	Promedio
Juan Duran	5.6 6.5 6.6	6,23
Lorena Perez	5.5 6.5	6,00
Alvaro Ruiz	3.5 5.0 5.6 2.3	4,10
Marcelo Pinto	5.5 5.5 6.6 6.3	5,98

- `mejor_alumno('INS-131')` retorna `'Juan Duran'`.
- Cada línea de `alumnos.txt` abre un segundo archivo.

Archivo inexistente

```
archivo = open('inmigrante.txt', 'r')    # FileNotFoundError
```

Archivo inexistente

```
archivo = open('inmigrante.txt', 'r')    # FileNotFoundError
```

- El nombre debe coincidir exactamente, con la extensión incluida.
- Python busca el archivo en la **carpeta desde donde se ejecuta** el programa.
- Lo más simple es guardar el archivo de datos junto al programa.

Escritura de archivos

Escritura con `write`

```
archivo = open('saludo.txt', 'w')
archivo.write('Hola\n')
archivo.write('Chao\n')
archivo.close()
```

saludo.txt

```
Hola
Chao
```

Escritura con `write`

```
archivo = open('saludo.txt', 'w')
archivo.write('Hola\n')
archivo.write('Chao\n')
archivo.close()
```

`saludo.txt`

```
Hola
Chao
```

- `write` recibe **un string** y lo escribe tal cual.
- `write` no agrega `\n`. El fin de línea se escribe explícitamente.
- En la consola, `write` muestra la cantidad de caracteres escritos (`5`). No es un error.

`write` recibe strings

```
salida = open('numeros.txt', 'w')
salida.write(25)                    # TypeError
salida.write(str(25) + '\n')
salida.write('{}\n'.format(25))
salida.close()
```

`write` recibe strings

```
salida = open('numeros.txt', 'w')
salida.write(25)                    # TypeError
salida.write(str(25) + '\n')
salida.write('{}\n'.format(25))
salida.close()
```

- Un número se convierte con `str()` o se inserta con `format`.
- Para escribir una lista, se une primero con `join`.

Un diccionario en un archivo

```
cuenta = {'Argentina': 1, 'EEUU': 3,
          'Venezuela': 1}
salida = open('paises.txt', 'w')
for pais in cuenta:
    linea = '{}:{}\n'.format(pais, cuenta[pais])
    salida.write(linea)
salida.close()
```

paises.txt

```
Argentina:1
EEUU:3
Venezuela:1
```

Un diccionario en un archivo

```
cuenta = {'Argentina': 1, 'EEUU': 3,
          'Venezuela': 1}
salida = open('paises.txt', 'w')
for pais in cuenta:
    linea = '{}:{}\n'.format(pais, cuenta[pais])
    salida.write(linea)
salida.close()
```

paises.txt

```
Argentina:1
EEUU:3
Venezuela:1
```

La plantilla fija el formato: llave, separador, valor y fin de línea.

El modo 'w' borra

```
archivo = open('inmigrantes.txt', 'w')
archivo.close()
```

El modo `'w'` borra

```
archivo = open('inmigrantes.txt', 'w')
archivo.close()
```

- `inmigrantes.txt` queda vacío. Basta abrirlo en modo `'w'`, aunque no se escriba nada.
- Los datos borrados no se recuperan.
- El archivo de salida lleva un nombre distinto al de entrada.

Agregar con 'a'

```
def agregar_ramo(rut, ramo, notas):
    textos = []
    for n in notas:
        textos.append(str(n))
    archivo = open(rut + '.txt', 'a')
    archivo.write('{},{}\n'.format(ramo, ' '.join(textos)))
    archivo.close()
    return len(notas)
```

Agregar con 'a'

```
def agregar_ramo(rut, ramo, notas):
    textos = []
    for n in notas:
        textos.append(str(n))
    archivo = open(rut + '.txt', 'a')
    archivo.write('{},{}\n'.format(ramo, ' '.join(textos)))
    archivo.close()
    return len(notas)
```

`agregar_ramo('150870162', 'DEP-010', [7.0, 4.5, 5.2])` agrega al final:

```
DEP-010,7.0 4.5 5.2
```

Agregar con 'a'

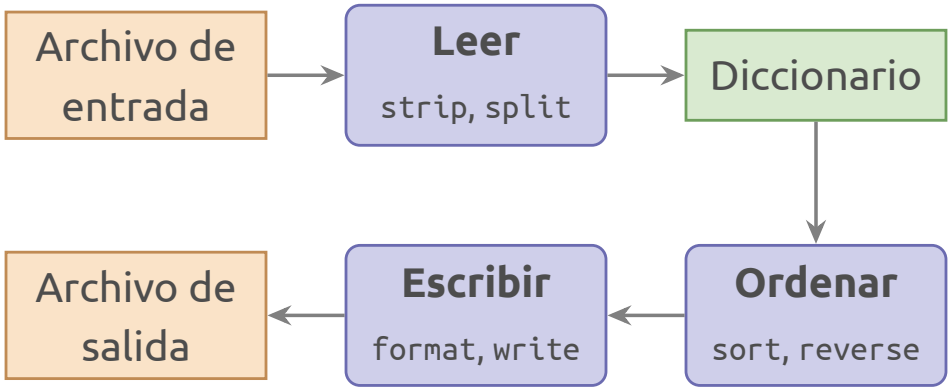
```
def agregar_ramo(rut, ramo, notas):
    textos = []
    for n in notas:
        textos.append(str(n))
    archivo = open(rut + '.txt', 'a')
    archivo.write('{},{}\n'.format(ramo, ' '.join(textos)))
    archivo.close()
    return len(notas)
```

`agregar_ramo('150870162', 'DEP-010', [7.0, 4.5, 5.2])` agrega al final:

```
DEP-010,7.0 4.5 5.2
```

Si la última línea del archivo no termina en `\n`, la línea nueva queda pegada a ella.

Leer, procesar y escribir



El ordenamiento se hace sobre una lista de listas construida desde el diccionario.

Aerolíneas por destino

`salidas-2019-08-09.csv` tiene los vuelos de un día, con formato `vuelo;destino;hora`. Las dos primeras letras del vuelo identifican la aerolínea.

```
UA7998;Tokyo;12:00
AA6991;Toronto;12:05
VS4667;Chicago;12:20
AA291;Toronto;12:59
AA3492;Chicago;12:20
UA23;Toronto;16:25
AA6250;Madrid;17:05
BA2492;Chicago;17:20
```

Aerolíneas por destino

`salidas-2019-08-09.csv` tiene los vuelos de un día, con formato `vuelo;destino;hora`. Las dos primeras letras del vuelo identifican la aerolínea.

```
UA7998;Tokyo;12:00
AA6991;Toronto;12:05
VS4667;Chicago;12:20
AA291;Toronto;12:59
AA3492;Chicago;12:20
UA23;Toronto;16:25
AA6250;Madrid;17:05
BA2492;Chicago;17:20
```

Se pide escribir `destinos.txt`, con la cantidad de aerolíneas **distintas** por destino, de mayor a menor.

```
Chicago:3
Toronto:2
Tokyo:1
Madrid:1
```

Lectura de las salidas del día

```
def agrupar(anio, mes, dia):
    nombre = 'salidas-{}-{}-{}.csv'.format(anio, mes, dia)
    archivo = open(nombre, 'r')
    lineas = {}
    for linea in archivo:
        vuelo, destino, hora = linea.strip().split(';')
        aerolinea = vuelo[:2]
        if destino not in lineas:
            lineas[destino] = []
        if aerolinea not in lineas[destino]:
            lineas[destino].append(aerolinea)
    archivo.close()
```

Lectura de las salidas del día

```
def agrupar(anio, mes, dia):
    nombre = 'salidas-{}-{}-{}.csv'.format(anio, mes, dia)
    archivo = open(nombre, 'r')
    lineas = {}
    for linea in archivo:
        vuelo, destino, hora = linea.strip().split(';')
        aerolinea = vuelo[:2]
        if destino not in lineas:
            lineas[destino] = []
        if aerolinea not in lineas[destino]:
            lineas[destino].append(aerolinea)
    archivo.close()
```

```
{'Tokyo': ['UA'], 'Toronto': ['AA', 'UA'],
 'Chicago': ['VS', 'AA', 'BA'], 'Madrid': ['AA']}
```

Escritura del ranking de destinos

```
    ranking = []
    for destino in lineas:
        ranking.append([len(lineas[destino]), destino])
    ranking.sort()
    ranking.reverse()
    salida = open('destinos.txt', 'w')
    for cantidad, destino in ranking:
        salida.write('{}:{}\n'.format(destino, cantidad))
    salida.close()
    return len(ranking)
```

Escritura del ranking de destinos

```
    ranking = []
    for destino in lineas:
        ranking.append([len(lineas[destino]), destino])
    ranking.sort()
    ranking.reverse()
    salida = open('destinos.txt', 'w')
    for cantidad, destino in ranking:
        salida.write('{}:{}\n'.format(destino, cantidad))
    salida.close()
    return len(ranking)
```

- `agrupar('2019', '08', '09')` escribe `destinos.txt` y retorna `4`.
- El nombre del archivo de entrada se construye con `format`.

Ruteo

Ruteo con strings

```
s = ' 7.0;6,5;4,0 \n'
partes = s.strip().split(';')
print(len(partes))
print(partes[1].replace(',', '.'))
print('-'.join(partes))
```

¿Qué imprime?

Ruteo con strings

```
s = ' 7.0;6,5;4,0 \n'
partes = s.strip().split(';')
print(len(partes))
print(partes[1].replace(',', '.'))
print('-'.join(partes))
```

¿Qué imprime?

```
3
6.5
7.0-6,5-4,0
```

Ruteo con strings

```
s = ' 7.0;6,5;4,0 \n'
partes = s.strip().split(';')
print(len(partes))
print(partes[1].replace(',', '.'))
print('-'.join(partes))
```

¿Qué imprime?

```
3
6.5
7.0-6,5-4,0
```

`replace` no cambia `partes[1]`, de modo que `join` une los strings originales.

Ruteo con un archivo

```
a = open('votos.txt', 'r')
d = {}
for l in a:
    n, v = l.strip().split(';')
    if n not in d:
        d[n] = 0
    d[n] += int(v)
a.close()
print(d)
```

votos.txt

```
Rojas;12
Soto;7
Rojas;5
Paz;3
```

¿Qué imprime?

Ruteo con un archivo

```
a = open('votos.txt', 'r')
d = {}
for l in a:
    n, v = l.strip().split(';')
    if n not in d:
        d[n] = 0
    d[n] += int(v)
a.close()
print(d)
```

votos.txt

```
Rojas;12
Soto;7
Rojas;5
Paz;3
```

¿Qué imprime?

```
{'Rojas': 17, 'Soto': 7, 'Paz': 3}
```

Errores frecuentes

Encuentre los tres errores

Sumar los kilómetros de todos los vuelos de `itinerario.txt`, cuyas líneas tienen el formato `NAC23:Santiago/Iquique/1600`.

```
archivo = open('itinerario.txt', 'w')
total = 0
for linea in archivo:
    codigo, ruta = linea.strip().split(':')
    origen, destino, km = ruta.split('/')
    total += km
print(total)
```

Encuentre los tres errores

(1) El modo `'w'` borra `itinerario.txt` y no permite leer. Debe ser `'r'`.

Encuentre los tres errores

(1) El modo `'w'` borra `itinerario.txt` y no permite leer. Debe ser `'r'`.

(2) `km` es un string, y `total += km` produce `TypeError`. Debe ser `total += int(km)`.

Encuentre los tres errores

(1) El modo `'w'` borra `itinerario.txt` y no permite leer. Debe ser `'r'`.

(2) `km` es un string, y `total += km` produce `TypeError`. Debe ser `total += int(km)`.

(3) Falta `archivo.close()` después del ciclo.

Encuentre los tres errores

(1) El modo `'w'` borra `itinerario.txt` y no permite leer. Debe ser `'r'`.

(2) `km` es un string, y `total += km` produce `TypeError`. Debe ser `total += int(km)`.

(3) Falta `archivo.close()` después del ciclo.

El primer error es el más grave: el archivo queda vacío antes de que el programa falle.

Encuentre el error

Contar los vuelos que salen a las 12:00.

```
archivo = open('salidas-2019-08-09.csv', 'r')
n = 0
for linea in archivo:
    vuelo, destino, hora = linea.split(';')
    if hora == '12:00':
        n += 1
archivo.close()
print(n)      # 0
```

Encuentre el error

Contar los vuelos que salen a las 12:00.

```
archivo = open('salidas-2019-08-09.csv', 'r')
n = 0
for linea in archivo:
    vuelo, destino, hora = linea.split(';')
    if hora == '12:00':
        n += 1
archivo.close()
print(n)      # 0
```

Falta `strip`. `hora` vale `'12:00\n'`, que nunca es igual a `'12:00'`.

Encuentre el error

Escribir un destino por línea.

```
salida = open('destinos.txt', 'w')
for destino in ['Chicago', 'Toronto', 'Tokyo']:
    salida.write(destino)
salida.close()
```

Encuentre el error

Escribir un destino por línea.

```
salida = open('destinos.txt', 'w')
for destino in ['Chicago', 'Toronto', 'Tokyo']:
    salida.write(destino)
salida.close()
```

`destinos.txt` contiene una sola línea:

```
ChicagoTorontoTokyo
```

Encuentre el error

Escribir un destino por línea.

```
salida = open('destinos.txt', 'w')
for destino in ['Chicago', 'Toronto', 'Tokyo']:
    salida.write(destino)
salida.close()
```

`destinos.txt` contiene una sola línea:

```
ChicagoTorontoTokyo
```

`write` no agrega el fin de línea. Debe ser `salida.write(destino + '\n')`.

Encuentre el error

Contar los registros y después mostrar los nombres.

```
archivo = open('inmigrantes.txt', 'r')
n = 0
for linea in archivo:
    n += 1
for linea in archivo:
    print(linea.strip().split(':')[1])
archivo.close()
```

Encuentre el error

Contar los registros y después mostrar los nombres.

```
archivo = open('inmigrantes.txt', 'r')
n = 0
for linea in archivo:
    n += 1
for linea in archivo:
    print(linea.strip().split(':')[1])
archivo.close()
```

El segundo `for` no imprime nada. El primer recorrido llega al final del archivo, y el segundo parte desde ahí.

Encuentre el error

Contar los registros y después mostrar los nombres.

```
archivo = open('inmigrantes.txt', 'r')
n = 0
for linea in archivo:
    n += 1
for linea in archivo:
    print(linea.strip().split(':')[1])
archivo.close()
```

El segundo `for` no imprime nada. El primer recorrido llega al final del archivo, y el segundo parte desde ahí.

Se hace todo en un solo recorrido, o se cierra y se vuelve a abrir el archivo.

Lo que usamos en esta unidad

SÍ

- `\n`, `\t` y `strip()`.
- `replace`, `split` y `join`.
- `format` con `{}` y `{:.1f}`.
- `open` y `close`, con los modos `'r'`, `'w'` y `'a'`.
- Recorrido de un archivo con `for`, línea a línea.
- `write` para escribir líneas.
- Archivos `.txt` y `.csv`, con uno o varios separadores.

NO

- `with open(...) as`.
- `read()`, `readline()` y `readlines()`.
- f-strings, como `f'{x}'`.
- La biblioteca `csv`.
- Otros tipos de archivo.

Ejercicios

Ejercicios de calentamiento

(a) Escriba una función que reciba un string con enteros separados por comas, como `'3,8,1'`, y retorne su suma.

(b) Escriba una función que reciba el nombre de un archivo y retorne su cantidad de líneas.

(c) Escriba una función que reciba el nombre de un archivo y una palabra, y retorne cuántas líneas contienen esa palabra.

(d) Escriba una función que copie un archivo en otro con todas las letras en mayúscula, y retorne la cantidad de líneas copiadas.

Ejercicio: ordenar el código

`perros.txt` tiene el formato `fecha;nombre;raza;problema;solucion`. Ordene e indente las líneas para que la función retorne el diccionario del ejemplo.

```
_, n, r, p, s = l.strip().split(';')
pe[n].append(s)
return pe
a = open(na)
a.close()
def leer_perros(na):
pe[n] = []
for l in a:
if n not in pe:
pe = {}
```

```
>>> leer_perros('perros.txt')
{'boby': ['collar de ahorque',
          'toque al costado'],
 'billy': ['toque al costado'],
 'willson': ['toque al costado',
             'toque al costado'],
 'firulais': ['SHHH', 'SHHH']}
```

Ejercicio: elecciones en Pythonia

`votaciones.txt` tiene líneas `rut:codigo` y `candidatos.txt` tiene líneas `codigo:nombre`. Un voto por un código que no está en `candidatos.txt` es nulo.

(a) `resultados(vot, can)` escribe `resultados.txt`, con el total de votos válidos y el porcentaje de cada candidato, de mayor a menor. Los empates pueden quedar en cualquier orden.

```
9173315-3:A1
19642628-0:A2
21933617-9:X27
173315-8:A2
516314-1:C2
```

`resultados.txt`

```
Total de votos: 3
Sebastian Guillier 66.7%
Alejandro Pinera 33.3%
Leopoldo Sharp 0.0%
...
```

Ejercicio: ingresos al país

Con `inmigrantes.txt` y la función `filtrar` de la clase:

(a) `contar_ingresos(nombre_archivo, año)`: retorna un diccionario que asocia cada país con la cantidad de personas que ingresaron ese año.

```
>>> contar_ingresos('inmigrantes.txt', 2019)
{'Argentina': 1, 'EEUU': 2, 'Venezuela': 1}
```

(b) `escribir_ingresos(nombre_archivo, año)`: escribe `ingresosAÑO.txt` con los países de mayor a menor cantidad, numerados desde 1, y retorna el total de personas.

```
>>> escribir_ingresos('inmigrantes.txt',
...                   2019)
4
```

`ingresos2019.txt`

```
1-EEUU:2
2-Venezuela:1
3-Argentina:1
```

Desafío: datos de salud

`datos.csv` tiene 308.854 líneas con el formato

```
estado_salud;chequeo;ejercicio;cardiaca;diabetes;artritis;genero;altura;peso;IMC
Poor;Within the past 2 years;No;No;No;Yes;Female;150.0;32.66;14.54
```

`imc_por_chequeo(nombre_archivo, estado)` escribe `estado-IMC.txt` con el IMC (índice de masa corporal) promedio de cada tipo de chequeo, de menor a mayor, y retorna la cantidad de chequeos.

```
>>> imc_por_chequeo('datos.csv',
...                 'Excellent')
5
```

```
Within the past 2 years: 25.88
Within the past 5 years: 25.91
5 or more years ago: 26.09
Never: 26.13
Within the past year: 26.17
```

Siglas

Sigla	Significado
CSV	*Comma separated values*, valores separados por comas
IMC	Índice de masa corporal
RUT	Rol único tributario